

docker

parte 1: fundamentos de Docker

De Git a Docker

- Con **Git** aprendimos a gestionar versiones del código.
- Ahora el siguiente paso en el despliegue: ¿cómo hacemos que ese código se ejecute igual en cualquier sitio?
- Esa es la pregunta que responde **Docker**.

El problema: "en mi máquina funciona"

- Tu aplicación funciona perfecto en tu portátil...
- ...pero falla en el servidor, o en el portátil de un compañero.
- Motivo habitual: **diferencias de entorno** (versión de lenguaje, librerías, configuración del sistema operativo...).

¿Qué es Docker?

Docker es una plataforma que permite empaquetar una aplicación junto con todo lo que necesita para funcionar (código, dependencias, configuración) en una unidad llamada **contenedor**.

- Proyecto de código abierto, nacido en 2013.
- Hoy en día, el estándar de facto para contenerizar aplicaciones.

¿Qué aporta Docker?

- El mismo entorno en desarrollo, pruebas y producción.
- Portabilidad: funciona igual en cualquier máquina con Docker instalado.
- Despliegues más rápidos y reproducibles.
- Aislamiento entre aplicaciones que corren en la misma máquina.

El problema real: estructura de equipos

- Equipo de **desarrollo**: escribir código, entregar funcionalidades rápido.
- Equipo de **operaciones**: desplegar y mantener los sistemas estables.
- Objetivos en tensión: velocidad de entrega vs. estabilidad.
- → Solución habitual: metodologías ágiles y cultura **DevOps**.

Cultura DevOps

- Enfoque que busca romper las barreras tradicionales entre Dev y Ops, fomentando una colaboración más estrecha.
-  No confundas la cultura DevOps con las **herramientas** que ayudan a lograrla.
- **Docker ayuda a lograr una cultura DevOps, pero Docker no es DevOps.**

Lectura recomendada

| *Accelerate. La ciencia del desarrollo Lean y DevOps* — Dra. Nicole Forsgren.

Contenedores vs. máquinas virtuales

Seguro que ya conoces las máquinas virtuales (VirtualBox, VMware...). Docker se parece, pero no es lo mismo.

Máquina virtual: virtualiza el hardware

- Un **hipervisor** reparte el hardware físico.
- Cada VM ejecuta un **sistema operativo invitado completo**.
- Aislamiento muy fuerte, pero "pesado": arranque lento, mucho consumo de disco/RAM.

Contenedor: virtualiza el sistema operativo

- Los contenedores **comparten el kernel** del sistema operativo host.
- No necesitan arrancar un SO completo: solo el proceso de la aplicación.
- Arranque casi instantáneo, mucho más ligeros.

Comparativa rápida

	Máquina virtual	Contenedor Docker
Virtualiza	Hardware completo	Sistema operativo (procesos)
Incluye SO propio	Sí	No (comparte el del host)
Arranque	Lento (minutos)	Rápido (segundos)
Peso	Pesado (GBs)	Ligero (MBs)
Aislamiento	Muy fuerte	Fuerte, pero comparte kernel

¿Se pueden combinar?

Sí: es muy habitual tener una **máquina virtual** (en un servidor físico o en la nube) que, dentro, ejecuta **varios contenedores Docker**.

Arquitectura de Docker: cliente y daemon

- **Docker client:** el comando `docker` que escribimos en la terminal.
- **Docker daemon** (`dockerd`): proceso en segundo plano que realmente gestiona contenedores, imágenes, redes y volúmenes.
- Cliente y daemon se comunican mediante una API (socket local o red).

Esquema básico

```
tú → docker run nginx → [Docker client] → API → [Docker daemon]
                                     |
                                     ¿tengo la imagen "nginx"?
                                     no → la pido a un registry
                                     sí → creo el contenedor
```

Registries: de dónde vienen las imágenes

- Un **registry** es un servidor donde se almacenan y distribuyen imágenes.
- El más conocido: **Docker Hub** (registry público oficial).
- Existen otros: GitHub Container Registry, Amazon ECR, registries privados de empresa...

Docker Hub

- Registro público con miles de imágenes listas para usar.
- **Imágenes oficiales:** mantenidas/revisadas por Docker o el propio proyecto (`ubuntu` , `nginx` , `python` , `node` ...).
- Se pueden **descargar imágenes públicas sin necesidad de cuenta**.
- Hace falta cuenta y `docker login` solo para **subir** imágenes propias.

Imágenes que usaremos en la asignatura

- `nginx` — servidor web.
- `mariadb` — base de datos relacional.
- `redis` — almacén clave-valor / caché.
- `php` — runtime de PHP.
- `mongo` — base de datos documental.

Cuidado con el tag `latest`

- Si no indicas tag, Docker usa `latest` por defecto.
- `latest` no significa "la mejor versión": es solo una etiqueta más, que puede apuntar a una versión distinta con el tiempo.
- **Buena práctica:** fija siempre una versión concreta.
 -  `ubuntu:22.04`
 -  `ubuntu` (usa `latest` sin que lo decidas tú)
 -  `ubuntu:latest` (explícito, pero mismo problema)

Imagen vs. contenedor (la diferencia clave)

- **Imagen:** plantilla de solo lectura (como una "foto" de un sistema con todo instalado).
- **Contenedor:** una instancia en ejecución (o parada) creada a partir de una imagen.

A partir de **una** imagen se pueden crear **muchos** contenedores distintos.

Analogía

- La **imagen** es como la clase de una aplicación (el molde).
- El **contenedor** es como un objeto/instancia creado a partir de esa clase.
- Puedes crear varios contenedores a partir de la misma imagen, cada uno independiente.

¿Qué mantiene vivo a un contenedor?

- Un contenedor sigue vivo mientras su **proceso principal** (PID1) siga en ejecución.
- En Linux, el **PID1** es el primer proceso que arranca el sistema (tradicionalmente `init / systemd`).

Contenedor = imagen + PID1

- Al hacer `docker run imagen comando`, ese `comando` se convierte en el **PID1** del contenedor.
- Si el PID1 termina, por el motivo que sea, **el contenedor se detiene automáticamente**.

Ejemplo: PID1 en la práctica

```
docker run -it ubuntu:22.04 bash
```

- Aquí, `bash` es el PID1 del contenedor.
- Si escribes `exit`, `bash` termina → el contenedor pasa a "Exited".
- Si en vez de `bash` lanzas `ubuntu:22.04 echo hola`, el PID1 es `echo`: termina al instante y el contenedor se para casi enseguida.

docker run — crear y arrancar un contenedor

```
docker run nginx
```

- Si la imagen `nginx` no está descargada, Docker la descarga primero.
- Crea un contenedor nuevo y lo arranca.

Opciones útiles de `docker run`

```
docker run -d --name miweb nginx      # segundo plano + nombre
docker run -it ubuntu bash            # sesión interactiva
docker run --rm hello-world           # se autoelimina al terminar
```

- `-d` : modo "detached" (segundo plano).
- `--name` : le pones un nombre fácil de recordar.
- `-it` : terminal interactiva.
- `--rm` : se borra solo al terminar (ideal para pruebas rápidas).

Si no usas `--name` ...

- Docker asigna automáticamente un **nombre aleatorio** (adjetivo + apellido de científico, p. ej. `silly_einstein`, `nostalgic_curie`).
- Por eso conviene usar `--name` : evita tener que buscar luego ese nombre con `docker ps`.

`docker ps` — ¿qué está en marcha?

```
docker ps          # solo contenedores en ejecución
docker ps -a       # todos, incluidos los parados
```

Muestra: ID, imagen, comando, fecha de creación, **estado**, puertos y nombre.

docker logs — ver qué dice un contenedor

```
docker logs miweb  
docker logs -f miweb      # en tiempo real  
docker logs --tail 50 miweb
```

Imprescindible para depurar: si algo falla, aquí suele estar la pista.

`docker images` y `docker pull`

```
docker images      # imágenes descargadas localmente  
docker pull nginx:1.25 # descarga sin crear contenedor
```

- `docker pull` solo descarga.
- `docker run` descarga (si falta) **y además** crea y arranca un contenedor.

Ciclo de vida de un contenedor

creado → en ejecución (running) → parado (exited) → eliminado

```
docker stop miweb      # parada ordenada
docker start miweb     # lo vuelve a arrancar (no crea uno nuevo)
docker restart miweb   # para y arranca de nuevo
docker rm miweb        # lo elimina definitivamente
```

`docker stop` vs `docker kill`

- `docker stop` : intenta parar el contenedor de forma ordenada, con un margen de tiempo.
- `docker kill` : lo detiene de forma inmediata y brusca.
- Usa `kill` solo cuando `stop` no responde.

`docker exec -it` — entrar en un contenedor vivo

```
docker exec -it miweb bash
docker exec -it miweb sh      # si no tiene bash (p.ej. alpine)
```

- A diferencia de `docker run`, **no crea** un contenedor nuevo.
- Abre una sesión dentro de un contenedor que **ya existe y está en ejecución**.
- Al salir con `exit`, el contenedor sigue funcionando con normalidad.

docker exec sin sesión interactiva

```
docker exec miweb ls /app  
docker exec miweb env
```

También sirve para lanzar un comando puntual sin abrir una terminal completa.

Repaso visual del flujo de hoy

<code>docker pull imagen</code>	→ la descargo
<code>docker run imagen</code>	→ creo y arranco un contenedor
<code>docker ps</code>	→ compruebo que está vivo
<code>docker logs</code>	→ reviso qué dice
<code>docker exec -it ... sh</code>	→ entro a inspeccionarlo
<code>docker stop / rm</code>	→ lo paro y limpio

Comandos que veremos hoy

- `docker run`, `docker ps`, `docker images`, `docker pull`
- `docker stop`, `docker start`, `docker restart`, `docker rm`
- `docker logs`, `docker exec -it`

A practicar

Primeros contenedores con Docker